

S - Responsabilidad Unica

Una clase debe tener una sola responsabilidad principal. Si una clase hace demasiadas cosas, se vuelve difícil de cambiar y entender. Por ejemplo, no conviene que una misma clase procese datos, los guarde y los muestre.

O - Abierto/Cerrado

El código debe estar abierto para agregar nuevas funciones, pero cerrado para modificar lo que ya funciona. Para eso se usan interfaces, clases abstractas o composición. Así se pueden agregar nuevos tipos sin romper el código anterior.

L - Sustitucion de Liskov

Una clase hija debe poder reemplazar a su clase padre sin que el programa falle. Si una subclase cambia el comportamiento esperado, la herencia está mal aplicada.

I - Segregacion de Interfaces

Es mejor tener interfaces pequeñas y específicas que una interfaz enorme. Una clase no debería estar obligada a implementar métodos que no necesita.

D - Inversion de Dependencia

Las clases deben depender de abstracciones, como interfaces, y no de clases concretas. Esto hace que el sistema sea más flexible y menos dependiente de detalles específicos.

Respuestas

1.

El código se vuelve más confuso, difícil de mantener y fácil de romper. Si se cambia una parte, puede afectar otras que no tienen relación. Además, se pierde reutilización porque todo queda mezclado en una sola clase.

2. ¿Como agregar nuevos vehiculos o herramientas sin modificar lo anterior?

Hay que usar interfaces o clases abstractas. Por ejemplo, crear una interfaz `Vehiculo` y que `Auto`, `Moto` o `Camion` la implementen. Así, si aparece un nuevo vehículo, solo se agrega una nueva clase sin tocar las anteriores.

3.

Un código limpio es fácil de leer, entender, modificar y probar. La alta cohesión permite que cada clase tenga una tarea clara. El bajo acoplamiento hace que las clases dependan menos entre sí. Esto evita que el programa se vuelva inmanejable cuando crece.